

2MMS80 - Statistical Learning Theory

Nearest neighbor classification and
handwritten digit classification

Spring 2026

1 Introduction

Sometimes simple ideas can be surprisingly good. This is the case with one of the oldest, but still rather popular *learning rule*, known as the *k-nearest neighbor* rule (abbreviated *k*-NN in this document). Consider the setting of supervised learning. Suppose you have a training data set $\{X_i, Y_i\}_{i=1}^n$, where $X_i \in \mathcal{X}$ and $Y_i \in \mathcal{Y}$, where $\mathcal{X}$ is a metric space (that is, a space endowed with a way to measure distances). As usual, our goal is to learn a prediction rule $f : \mathcal{X} \rightarrow \mathcal{Y}$ that is able to do “good” predictions on unseen data.

The idea of *k*-NN is remarkably simple. Given a point $x \in \mathcal{X}$ for which we want to make a prediction, we simply look for the *k* “closest” points in the training set and make a prediction based on a majority vote (classification) or average (regression) of the neighbor labels. That is as simple as that. Computationally this might seem cumbersome, particularly for large datasets. But one can use clever computational tricks to ensure this can be done quickly.

In this assignment, which is divided in two parts, you will: (i) get a first-hand experience with this method by implementing it and choosing a good set of tunable parameters in a sound way; (ii) analyze the performance of this method in some generality and get a better understanding why it is sensible.

To make the explanation more concrete let us consider the problem of handwritten digit classification (which is the topic of part I): given a low resolution image of a handwritten digit we would like to classify it as one of the digits in $\{0, 1, \dots, 9\}$. More specifically our images have 28×28 pixels, each pixel taking values in $\{0, 1, 2, \dots, 255\}$. Therefore $\mathcal{X} = \{0, 1, \dots, 255\}^{28 \times 28}$ and $\mathcal{Y} = \{0, 1, \dots, 9\}$.

2 The k -NN rule

Let $d : \mathcal{X} \times \mathcal{X} \rightarrow [0, +\infty)$ be a metric¹ in $\mathcal{X}$. Let $x \in \mathcal{X}$ be an arbitrary point in $\mathcal{X}$ and consider the re-ordering of each pair of the training data as

$$(X_{(1)}(x), Y_{(1)}(x)), (X_{(2)}(x), Y_{(2)}(x)), \dots, (X_{(n)}(x), Y_{(n)}(x)) ,$$

so that

$$d(x, X_{(1)}(x)) \leq d(x, X_{(2)}(x)) \leq \dots \leq d(x, X_{(n)}(x)) . \quad (1)$$

Note that the ordering depends on the specific point x (hence the cumbersome notation) and might not be unique. In that case we can break ties in some pre-defined way (e.g., if two points are at equal distance from x the point that appears first in the original dataset will also appear first in the ordered set). The k -NN rule (for classification) is defined as

$$\hat{f}_n(x) = \arg \max_{y \in \mathcal{Y}} \left\{ \sum_{i=1}^k \mathbf{1} \{Y_{(i)}(x) = y\} \right\} . \quad (2)$$

In other words, just look among the k -nearest neighbors and choose the class that is represented more often. Obviously, there might be situations where two (or more) classes appear an equal number of times. In such situations a tie-breaking rule needs to be specified.

The performance of the method described above hinges crucially on the choice of two parameters: k , the number of neighbors used for prediction and; $d : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$, the distance metric used to define proximity of two points in the feature space. There are many possible choices for d , and a naïve but sensible starting point is to consider the usual Euclidean distance: if $x, y \in \mathbb{R}^l$ then the Euclidean distance is simply given by $\sqrt{\sum_{i=1}^l (x_i - y_i)^2}$.

3 The MNIST dataset

The MNIST dataset² is a classical dataset frequently used to demonstrate machine learning methods, and is still often used as a benchmark to demonstrate methodologies. This dataset is provided as *comma-separated value* (csv) files in CANVAS. The training set `MNIST_train.csv` consists of 60000 images of handwritten digits and the corresponding label (provided by a human expert). The test set `MNIST_test.csv` consists of 10000 images of handwritten digits and the corresponding labels. In addition, in CANVAS you will also find two smaller training and test sets, `MNIST_train_small.csv` (3000 examples) and `MNIST_test_small.csv` (1000 examples). These will be used for a large part of the assignment, to avoid the struggles associated with large datasets and to test out your implementations.

¹A metric or distance is a function that must satisfy the following properties: (i) $\forall x, y \in \mathcal{X} \ d(x, y) = 0 \iff x = y$; (ii) $\forall x, y \in \mathcal{X} \ d(x, y) = d(y, x)$ (symmetry); (iii) $\forall x, y, z \in \mathcal{X} \ d(x, y) \leq d(x, z) + d(z, y)$ (triangle inequality).

²See <http://yann.lecun.com/exdb/mnist/> for more details and information.



Figure 1: First five examples from `MNIST_train.csv` and the corresponding labels provided by a human expert.

The format of the data is as follows: each row in the `.csv` file has 785 entries and corresponds to a single example. The first entry in the row is the “true” label, in $\mathcal{Y} = \{0, 1, \dots, 9\}$ and the 784 subsequent entries encode the image of the digit – each entry corresponding to a pixel intensity, read in a lexicographical order (left-to-right then top-to-bottom). Pixel intensities take values in $\{0, 1, \dots, 255\}$. The Matlab function `showdigit.m` in CANVAS will take as input a row of this data and display the corresponding digit image. Figure 3 shows several examples from `MNIST_train.csv`.

Ultimately the goal is to minimize the probability of making errors. For the purposes of this assignment we will use simply the 0/1 loss. This means that the empirical risk is simply the average number of errors we make. If $\{X'_i, Y'_i\}_{i=1}^m$ denotes the pairs of features/labels in a test set and $\{\hat{Y}'_i\}_{i=1}^m$ denotes the corresponding inferred labels by the k -NN rule then the empirical risk on the test set is given by $\frac{1}{m} \sum_{i=1}^m \mathbf{1}\{\hat{Y}'_i \neq Y'_i\}$.

PART I - Computational Assignment

The goal of the first part of the assignment is to implement a nearest neighbor classifier, assess its performance on a specific prediction task, and understand the impact of choices such as a the number of neighbors and distance metric used. Although it is quite instructive to do such an implementation without resorting to specialized libraries and implementations that do not target specifically nearest neighbor methods, this is not a hard requirement. For this assignment you are allowed to use any language or command interpreter (preferably a high-level language, but not necessarily so). You will not be assessed on your code, but rather on your choices and corresponding justification.

You should prepare a report (in English) and upload it via Feedback Fruits (link in CANVAS). The report should be self-contained, and you should pay attention to the following points:

- The report should feature an introduction, explaining the problem and methodology.
- Use complete sentences: there should be a coherent story and narrative - not simply numerical answers to the questions without any criticism or explanation.

- Pay attention to proper typesetting. Use a spelling checker. Make sure figures and tables are properly typeset.
- **It is very important that for you to have a critical attitude, and comment on the your choices and results.**
- Although you will not be assessed on the code itself, you are ultimately responsible for all the implementation choices you made. Therefore, in addition to the report, you are asked to also submit a separate .pdf file with the code/script you developed and used.
- Use of LLM's and AI tools to generate the report is not allowed!!! Any other use of AI tools should be clearly acknowledged in the report. Suspected misuse of AI tools will be reported to the examination committee.

The report for part I should be submitted as a single .pdf file. In addition, submit a separate .pdf file with the code/script you used. In your report you should do the following experiments and answer the questions below.

- Implement your routine of k -NN neighbors classification, and use it in the context of classification of hand-written digits. Specifically, use as training data `MNIST_train_small.csv` and report on its error rate (i.e., the empirical loss) when predicting the labels in both the training and test sets (respectively `MNIST_train_small.csv` and `MNIST_test_small.csv`). For this question use the simple Euclidean distance. Specifically, make a table of results for $k \in \{1, \dots, 20\}$, plot your the empirical training and test loss as a function of k , and comment on your results. **Crucially, explain how ties are broken, namely those in Equation (1) and even more importantly in Equation (2).**
- Obviously the choice of the number of neighbors k is crucial to obtain good performance. This choice must be made WITHOUT LOOKING at the test dataset. Although one can use data-agnostic rules-of-thumb, a possibility is to use cross-validation. *Leave-One-Out Cross-Validation* (LOOCV) is extremely simple in our context. Implement LOOCV to estimate the risk of the k -NN rule for $k \in \{1, \dots, 20\}$. Report these LOOCV risk estimates³ on a table and plot them as well the empirical loss on the test dataset (that you obtained in (a)). Given your results, what would be a good (and non-clairvoyant) choice for k ? Comment on your results.
- Obviously, the choice of distance metric also plays an important role. Consider a simple generalization of the Euclidean distance, namely ℓ_p distances (also known as Minkowski distances). For $x, y \in \mathbb{R}^l$ define

$$d_p(x, y) = \left(\sum_{i=1}^l |x_i - y_i|^p \right)^{1/p},$$

³Recall that these estimates use only the information on the training dataset.

where $p \geq 1$. Use leave-one-out cross validation to simultaneously choose a good value for $k \in \{1, \dots, 20\}$ and $p \in \{1, 2, \dots, 15\}$.

- d) **(this question is more open)** Building up on your work for the previous questions suggest a different distance metric or some pre-processing of the data that you consider appropriate to improve the performance of the k -NN method. Note that, any choices you make should be done solely based on the training data (that is, do not clairvoyantly optimize the performance of your method on the test data). Clearly justify ALL the choices made and describe the exact steps you took. Someone reading your report should be able to replicate your results.

Now that you implemented and tested your methodologies in a smaller scale, let us see how these methods scale to the full datasets. For the remaining questions you will use the full MNIST training and test sets.

- e) Make use of either the Euclidean distance or d_p with your choice of p in part (c) (use only one or the other). Determine a good value for k using leave-one-out cross validation when considering the full training set (60000 examples). Was your implementation able to cope with this large amount of data? Did you have to modify it in any way? If so, explain what you did. What is the risk estimate you obtain via cross-validation?
- f) Using the choice of k in part (e) compute the loss of your method on the test set provided. How does this compare with the cross-validation estimate you computed in (e)? Would you choose a different value for k had you been allowed to look at the test dataset earlier?
- g) **Optional:** each training example is currently a high-dimensional vector. A very successful idea in machine learning is that of dimensionality reduction. This is typically done in an unsupervised way - feature vectors are transformed so that most information is preserved, while significantly lowering their dimension. A possibility in our setting is to use Principal Component Analysis (PCA) to map each digit image to a lower dimensional vector. There is an enormous computational advantage (as computing distances will be easier) but there might be also an advantage in terms of statistical generalization. Use this idea in our setting, and choose a good number of principal components to keep in order to have good accuracy (again, this choice should be solely based on the training data). Document clearly all the steps of your procedure. You are allowed to use an existing implementation of PCA or related methods.

IMPORTANT: if for some reason you are unable to make things work for the large datasets, use instead for the training data the first 20000 rows of `MNIST_train.csv` and for testing the first 5000 rows of `MNIST_test.csv`.